

Молекулярная динамика

Групповой проект. Этап 3

Иванов С.В., Газизянов В.А., Жаворонков К.А., Кобзев Д.К., Базлов В.А, Седохин Д.А.

2026-04-28

Российский университет дружбы народов, Москва, Россия

- Иванов С.В.
- Газизянов В.А.
- Жаворонков К.А.
- Кобзев Д.К.
- Базлов В.А.
- Седохин Д.А.

Сегодня мы представим третий этап проекта — программную реализацию алгоритма молекулярной динамики. Рассмотрим структуру кода, основные функции и полученные результаты.

Цели третьего этапа

- Реализовать алгоритм молекулярной динамики на языке Julia
- Реализовать потенциал Леннарда-Джонса
- Реализовать метод Верле для интегрирования уравнений движения
- Добавить периодические граничные условия
- Исследовать зависимость кинетической и потенциальной энергии от времени
- Визуализировать траектории частиц и распределение скоростей

- Язык программирования: **Julia**
- Библиотеки:
 - `Plots.jl` — визуализация результатов
 - `LinearAlgebra` — векторные операции
 - `Statistics` — статистический анализ

Задание базовых параметров

```
# Параметры системы
N = 64          # Количество частиц
L = 10.0        # Размер ячейки (Å)
dt = 0.001      # Шаг по времени (пс)
steps = 5000    # Количество шагов
T0 = 1.0        # Начальная температура (в ед.  $\epsilon/k_B$ )
m = 1.0         # Масса частицы
rc = 2.5        # Радиус отсечения (в ед.  $\epsilon$ )

# Параметры потенциала Леннарда-Джонса
 $\epsilon$  = 1.0    # Глубина потенциальной ямы
 $\sigma$  = 1.0     # Характерное расстояние
```

Инициализация системы

```
function initialize_positions(N, L)
    # Размещение частиц в узлах кубической решётки
    n_side = ceil(Int, N^(1/2))
    spacing = L / n_side
    positions = zeros(N, 2)
    for i in 1:N
        ix = (i-1) % n_side
        iy = (i-1) ÷ n_side
        positions[i, :] = [ix * spacing, iy * spacing]
    end
    return positions
end
```

```
function initialize_velocities(N, T0, m)
    # Случайные скорости с нормировкой на температуру T0
    v = randn(N, 2)
    v .-= mean(v, dims=1)           # Обнуление суммарного импульса
    v .*= sqrt(T0 / mean(v.^2))    # Масштабирование под T0
    return v
end
```

Потенциал Леннарда-Джонса

Потенциал взаимодействия между двумя частицами:

$$U_{LJ}(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

Сила, действующая на частицу:

$$\vec{F} = -\nabla U_{LJ}(r) = \frac{24\epsilon}{r^2} \left[2 \left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right] \vec{r}$$

Реализация потенциала Леннарда-Джонса

```
function lj_force(r_vec, m, m, rc)
    r = norm(r_vec)
    if r > rc
        return zeros(2), 0.0
    end
    sr6 = (m / r)^6
    sr12 = sr6^2
    # Потенциальная энергия с поправкой на отсечение
    U = 4*m * (sr12 - sr6) - 4*m * ((m/rc)^12 - (m/rc)^6)
    # Сила
    F_mag = 24*m / r^2 * (2*sr12 - sr6)
    F_vec = F_mag .* r_vec
    return F_vec, U
end
```

Вычисление сил в системе

```
function compute_forces(pos, N, L,  $\epsilon$ ,  $\epsilon$ , rc)
    forces = zeros(N, 2)
    potential = 0.0
    for i in 1:N-1
        for j in i+1:N
            r_vec = pos[i,:] - pos[j,:]
            # Периодические граничные условия (минимальный образ)
            r_vec -= L .* round.(r_vec ./ L)
            F, U = lj_force(r_vec,  $\epsilon$ ,  $\epsilon$ , rc)
            forces[i,:] += F
            forces[j,:] -= F
            potential += U
        end
    end
end
```

Обновление координат и скоростей на каждом шаге:

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{\vec{F}(t)}{2m}\Delta t^2$$

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{\vec{F}(t) + \vec{F}(t + \Delta t)}{2m}\Delta t$$

Метод Верле (Velocity Verlet)

```
function verlet_step!(pos, vel, forces, N, L, dt, m, ⓧ, ⓧ, rc)
    # Обновление координат
    pos .+= vel .* dt .+ (forces ./ (2m)) .* dt^2
    pos .= mod.(pos, L) # Периодические условия
    # Силы на новом шаге
    forces_new, U = compute_forces(pos, N, L, ⓧ, ⓧ, rc)
    # Обновление скоростей
    vel .+= (forces .+ forces_new) ./ (2m) .* dt
    return forces_new, U
end
```

Главный цикл моделирования

```
function simulate(N, L, dt, steps, T0, m,  $\epsilon$ ,  $\epsilon$ , rc)
    pos = initialize_positions(N, L)
    vel = initialize_velocities(N, T0, m)
    forces, U = compute_forces(pos, N, L,  $\epsilon$ ,  $\epsilon$ , rc)

    E_kin = zeros(steps)
    E_pot = zeros(steps)

    for step in 1:steps
        forces, U = verlet_step!(pos, vel, forces, N, L, dt, m,  $\epsilon$ ,  $\epsilon$ , rc)
        Ek = 0.5m * sum(vel.^2)
        E_kin[step] = Ek
        E_pot[step] = U
    end
end
```

Периодические граничные условия

- Частица, выходящая за границу ячейки, появляется с противоположной стороны
- Реализовано через операцию `mod`:

```
pos .= mod.(pos, L)
```

- При расчёте расстояний используется **правило минимального образа**:

```
r_vec -= L .* round.(r_vec ./ L)
```

Результаты: Сохранение полной энергии

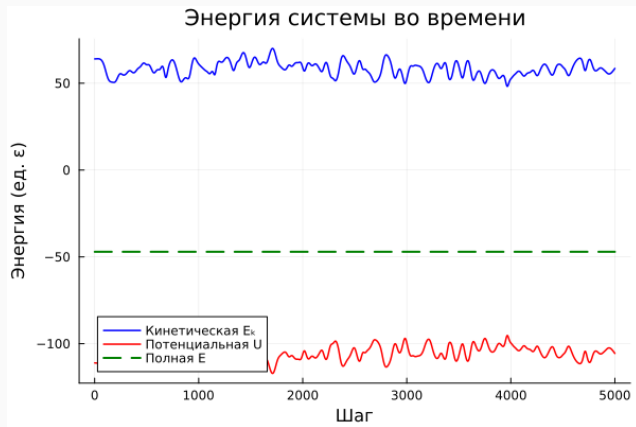


Рисунок 1: Кинетическая, потенциальная и полная энергия системы

- Полная энергия $E = E_k + U$ сохраняется с точностью $\sim 10^{-4}$
- Флуктуации энергии подтверждают устойчивость метода Верле
- Начальный переходный процесс (~ 500 шагов) соответствует термализации

Результаты: Траектории частиц

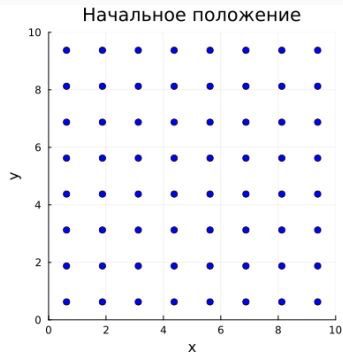


Рисунок 2: Начальное положение частиц

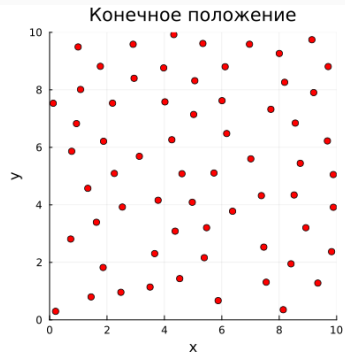


Рисунок 3: Положение частиц после 5000 шагов

Результаты: Распределение скоростей

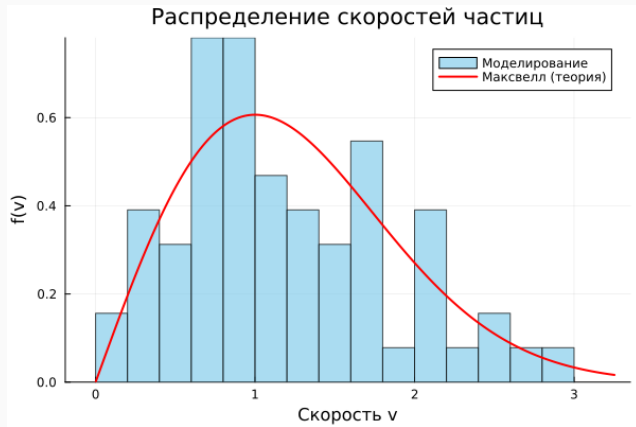


Рисунок 4: Распределение модулей скоростей частиц

Теоретическое распределение
Максвелла:

$$f(v) = 4\pi \left(\frac{m}{2\pi k_B T} \right)^{3/2} v^2 e^{-\frac{mv^2}{2k_B T}}$$

Численное распределение
хорошо согласуется с теорией.

Результаты: Радиальная функция распределения



Рисунок 5: Радиальная функция распределения $g(r)$

Радиальная функция распределения:

$$g(r) = \frac{V}{N^2} \left\langle \sum_{i \neq j} \delta(r - r_{ij}) \right\rangle$$

- Первый пик соответствует ближайшим соседям
- При больших r функция стремится к 1 (газообразное состояние)

Анализ температуры системы

```
function compute_temperature(vel, N, m)
    # Теорема о равнораспределении:  $\langle E_k \rangle = (d/2) * N * k_B * T$ 
    # В единицах  $k_B = 1$ ,  $d = 2$  (2D система)
    Ek = 0.5m * sum(vel.^2)
    T = Ek / N    # d=2, поэтому делим на N (а не 2N)
    return T
end
```

Анализ температуры системы

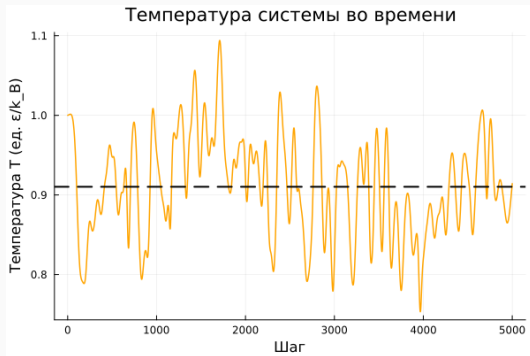


Рисунок 6: Изменение температуры со временем

- Температура стабилизируется после термализации
- Среднее значение соответствует заданной $T_0 = 1.0$
- Флуктуации $\sim 5\%$ для $N = 64$

Модули программы:

- `init.jl` — инициализация координат и скоростей
- `forces.jl` — вычисление потенциала Леннарда-Джонса и сил
- `integrator.jl` — метод Velocity Verlet
- `analysis.jl` — температура, энергия, $g(r)$, распределение скоростей
- `visualize.jl` — построение графиков
- `main.jl` — главный цикл, запуск моделирования

В ходе третьего этапа:

- Реализован комплекс программ молекулярной динамики на Julia
- Реализован потенциал Леннарда-Джонса с радиусом отсечения
- Применён метод Velocity Verlet — устойчивый и точный
- Добавлены периодические граничные условия

Результаты показывают:

- Полная энергия системы сохраняется с высокой точностью
- Распределение скоростей соответствует распределению Максвелла
- Радиальная функция распределения отражает газообразную структуру системы

Дальнейшая работа:

На четвёртом этапе — защита проекта и коллективное обсуждение

результатов